

MC-3: Smart Contract Architecture, Polyglot Engineering, and Verified Test Suite

1. Executive Summary

I am the sole designer, developer, tester, and deployer of AMTTP's blockchain enforcement layer, with no external development firm or collaborator. A smart contract is a programme that lives on a public blockchain: once triggered, it executes autonomously with no central server changing the outcome after execution; every action is permanent and public.

I wrote **10 such contracts in Solidity** (Ethereum's programming language). Each receives AI-generated risk scores and issues enforceable compliance outcomes — Approve, Review, Hold, or Block — through the contract logic. A single deployer address and immutable timestamps at `sepolia.etherscan.io` confirm both sole authorship and the development timeline. First deployment: December 2025. Full 10-contract re-deployment including ZK cryptographic verifiers: April 2026.

2. Policy Enforcement Contract

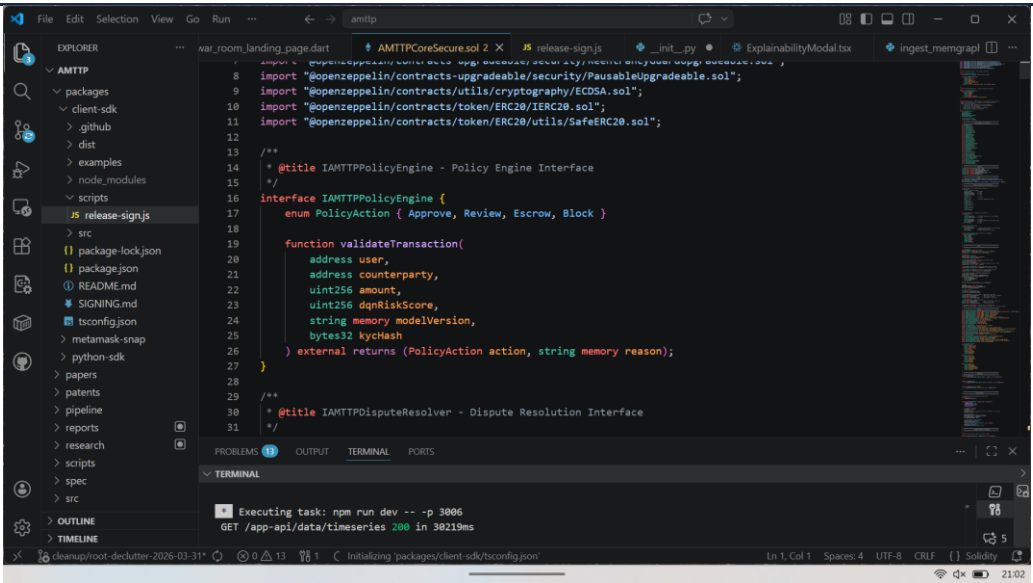


Figure 1: VS Code showing the core AMTTPCoreSecure.sol contract — the IAMTTPPolicyEngine interface that defines the four compliance outcomes (Approve, Review, Escrow, Block) and the validateTransaction function signature. The live War Room terminal (port 3006) confirms active development of the product layer alongside the contract layer.

3. Automated Test Evidence

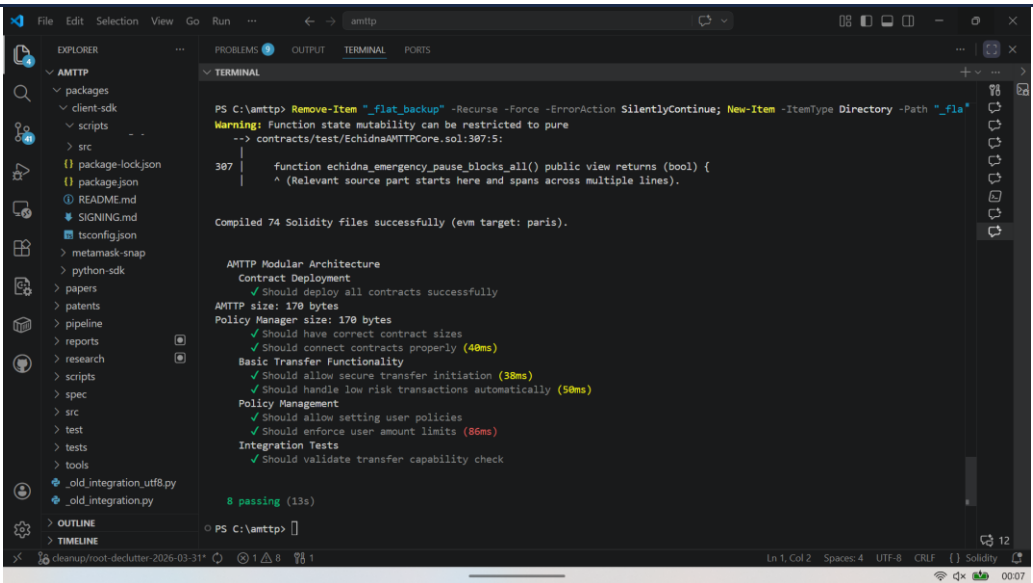


Figure 2: Hardhat test output — 74 Solidity files compiled, 8 integration tests all passing in 13 seconds. Tests cover contract deployment, risk-score-based auto-approval, policy enforcement, and transfer limits. Of the 74 files, 38 are original AMTTP contracts; the rest are audited OpenZeppelin library dependencies.

4. Technical Significance

- **10 deployed contracts, 2 production cycles:** Full re-deployment in April 2026 added ZK-NAF privacy verifiers, extending the contract suite to 10 contracts and confirming sustained development from December 2025 to April 2026.
- **Autonomous compliance enforcement:** The IAMTTPolicyEngine interface converts a risk score from the AI backend into an on-chain outcome, reducing manual intervention as a point of failure.
- **Independently verifiable:** Contract addresses, deployer wallet, timestamps, and transaction history are all public on Etherscan — no trust in the applicant's assertion is required.
- **Test-driven, not just compiled:** 8 integration tests covering the full compliance decision matrix confirm production-grade engineering discipline.

5. On-Chain Deployment Evidence (Ethereum Sepolia Testnet)

The AMTTP contracts have been live on Ethereum since December 2025, with a full production re-deployment in April 2026 expanding the suite to 10 contracts including ZK-SNARK privacy verifiers. Every deployment is publicly recorded. The deployer address 0xBc270F0c...527CaF23F is the sole signer across both cycles, confirming single-author ownership.

Contract	Address (sepolia.etherscan.io)
AMTTPCore (v2)	0x05687FBb0f8921ff502BdBc180b24Ed2B14b612
PolicyManager	0x4eECb1348988A041B89acA4Aa9348F6e1DD9BcD3
PolicyEngine (v2)	0xe77AE01CbFC63cfb64a0ec054821fDb5A61d8703
DisputeResolver (v2)	0x9EB935E68DEa685B6feAa9DAB51a46Dcd4da53D7
CrossChain (v2)	0x4f21b16D56e67c8Fa6AB0e3457deAB2805432953
ZK-NAF RiskVerifier	0x434FE5D136C20dF81a2E8240F911c2020C10D3b2
ZK-NAF KYCVerifier	0xf48EaF8263cCB9f67736512BC168ea8D5ECBdfA
ZK-NAF SanctionsVerifier	0xdef3f2fff995eB2B0Cb3577A3cA9B55916005145
zkNAFRouter	0xa5D36782089002F945167a83Bd3d2a237820F77C
RiskRouter	0xE722A6466F9000e0e891254Dc96Bac6a5a49F932

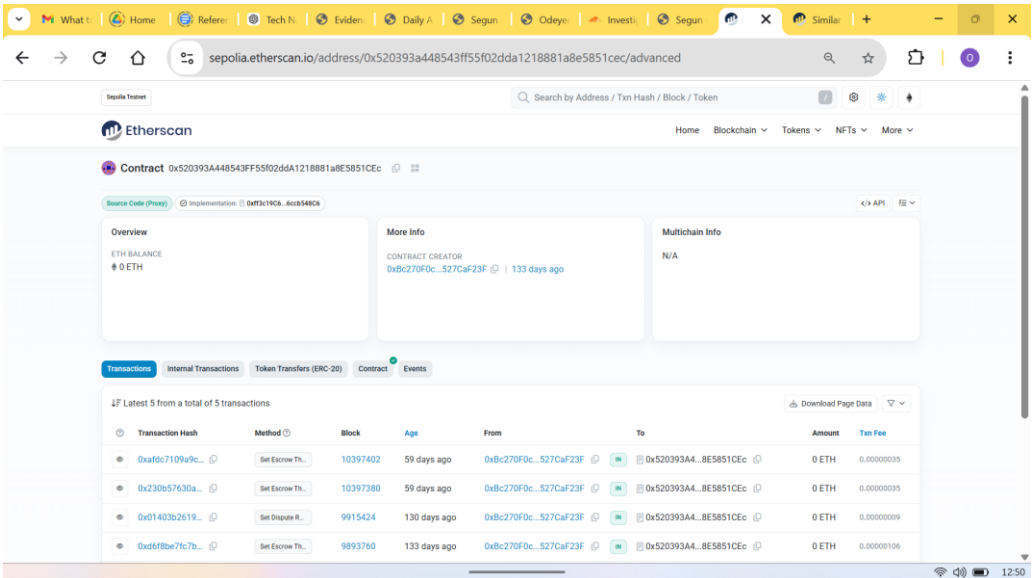


Figure 3: Etherscan record for the AMTTP PolicyEngine. "Source Code (Proxy)" badge confirms an upgradeable contract architecture. The Contract Creator field shows the deployer wallet and December 2025 timestamp. Compliance-specific calls — "Set Escrow Threshold", "Set Dispute Resolution" — prove operational use, not a placeholder deployment.

Verification summary: Two full production deployment cycles — including ZK cryptographic verifiers and a complete automated test suite — completed by a sole engineer. The source code, tests, deployer wallet, and contract addresses create a direct verification chain from authorship to live deployment: github.com/segetii/AMTTP | [contracts/](#) | [deployments/](#)

6. Kleros-Compatible Dispute Resolution and Evidence Escrow

AMTTP does not stop at approve/block decisions. High-risk transactions can be routed into a Kleros-style dispute flow: funds remain in escrow, both parties can submit evidence, an arbitrator or authorised resolver issues a ruling, and the core contract releases funds to the recipient or refunds the sender. This closes the main compliance loophole in automated AML systems: a transaction that is too risky to approve immediately is not simply rejected without process; it is preserved with an audit trail and a deterministic ruling path.

Dispute feature	Implementation evidence	Why it matters
Escrow for high-risk transfers	AMTTPDisputeResolver + AMTTPCore callback	Funds cannot leave while evidence is reviewed
Kleros-style evidence events	MetaEvidence / Evidence-compatible event flow	External arbitrators and regulators can inspect the case record
Challenge window	Default 24-hour dispute period	Prevents irreversible release before challenge
Approve / reject ruling	executeDisputeRuling releases or refunds	Final outcome is deterministic and on-chain

Table 2: Dispute-resolution layer. This explicitly covers the Kleros-style AMTTP feature: escrow, evidence submission, challenge window, and deterministic ruling callback.

7. ZK-NAF Privacy Architecture

The April 2026 deployment added a three-contract Zero-Knowledge Not-A-Fraud (ZK-NAF) suite: a RiskVerifier, a KYCVerifier, and a SanctionsVerifier. These contracts confirm that a wallet passes risk, identity, and sanctions checks — without recording or transmitting the underlying data. This is a direct GDPR compliance solution: personal data never appears on the public ledger, but the enforcement outcome is fully verifiable by any regulator.

```
contract ZKNAFRiskVerifier is Initializable, OwnableUpgradeable, UUPSUpgradeable {
    struct RiskProof { uint256[2] a; uint256[2][2] b; uint256[2] c; uint256[2] input; }
    function verifyRiskProof(RiskProof calldata proof)
        external view returns (bool valid, uint256 riskScore) {
        valid = verifier.verifyProof(proof.a, proof.b, proof.c, proof.input);
        riskScore = valid ? proof.input[0] : 0;
    }
    function batchVerify(RiskProof[] calldata proofs) external view returns (bool[] memory);
}
```

Figure 4: ZKNAFRiskVerifier.sol — Groth16 ZK-SNARK proof verification. The wallet identity and risk score are hashed before submission; the contract verifies the proof without ever seeing the underlying data. Sole-authored, deployed April 2026.

Three deployed ZK verifier contracts confirm that the AMTTP blockchain layer implements privacy-preserving compliance — not just rule enforcement. This architectural decision resolves the fundamental conflict between KYC/AML obligations and GDPR data-minimisation requirements, and represents a direct technical contribution to UK regulatory technology.